Orchestrating Teams of Agents

Last modified: 2026-09-01 23:07:25 (PDT)

A single coding agent works one problem at a time. *Orchestration* is the step up from that: running several agents at once and coordinating their work. This chapter explains when orchestration is worth the added cost, describes what our lab already uses for it, and evaluates three outside "agent orchestrator" projects that lab members have asked about.

> ⚠ A fast-moving, opinionated snapshot
>
> As of early 2026, this space is changing weekly. The tools below are young, and the star counts, version numbers, and "experimental" labels quoted here were true when this chapter was written (August 2026) and will drift. The verdicts are the lab's current opinion, not settled fact. Re-check before acting on any of them.

1 When Orchestration Helps

Orchestration pays off when work splits into pieces that can run at the same time without waiting on each other. The strongest cases are:

- **Research and review**: several agents investigate different angles at once, then compare and challenge each other's findings.
- **Independent implementation**: each agent owns a separate module, file set, or task.
- **Adversarial verification**: agents test competing hypotheses in parallel and converge faster than one agent anchoring on its first guess.

Orchestration is not free. Every extra agent has its own context window and spends tokens independently, so cost grows with the number of agents, and coordination overhead grows too. For sequential work, edits to the same file, or tasks with many dependencies, a single agent is cheaper and simpler. The question for any orchestration tool is not "is it impressive?" but "what does it add to what we already do?"

2 What We Already Use

Our lab already orchestrates agents, using Claude Code plus the conventions in our `ai-config` repository. The pieces are:

- **Subagents**: a session spawns focused helper agents for search, review, or verification. Each has its own context window and reports its result back to the main session.
- **The `Workflow` fan-out tool**: a deterministic script that spreads work across many agents in parallel (for example, one reviewer per dimension of a pull request), with token budgets and verification stages built in.
- **Git worktrees**: isolated checkouts that let parallel sessions edit a repository without clobbering each other.
- **Hooks and permissions**: guardrails that gate risky actions, scope tool access, and enforce lab rules mechanically.
- **Reusable GitHub Actions workflows** (in `Morrison-Lab/gha`): the automated review-and-merge workflow for pull requests.

This baseline matters because it is the yardstick for everything below. A new orchestrator is useful to us only if it does something this stack does not already do well.

3 Claude Code Agent Teams

Agent Teams (Anthropic 2026) is the built-in Anthropic feature for coordinating several Claude Code sessions. One session acts as the **lead**, and it spawns **teammates**, each a full, independent Claude Code session with its own context window. Teammates coordinate through a shared task list and a mailbox, and, unlike subagents, they can message each other directly rather than only reporting back to the lead.

The intended uses match the strong cases in Section 1: parallel code review, competing-hypothesis debugging, new modules owned by different teammates, and changes that span several layers of a codebase. As of August 2026 the feature is **experimental** and off by default; it is enabled with the `CLAUDE_CODE_EXPERIMENTAL_AGENT_TEAMS` environment variable. It carries real limits: teammates do not survive session resumption, a session has exactly one team, teammates cannot spawn their own teammates, and token use is much higher than a single session.

> Useful to us? Yes, worth trying
>
> Agent Teams is a native extension of the tool we already live in, so it is the lowest-friction option here. It complements, rather than replaces, our subagents, `Workflow` fan-out, and worktree parallelism: teammates that talk to each other suit adversarial review and debugging, where our subagents (which only report back) are weaker. Treat it as a feature

to experiment with on research and review tasks, not a platform to adopt, and mind the token cost and the experimental limits.

4 Inflexa

Inflexa (Inflexa 2026) calls itself "the open-source orchestrator for computational biology." It is a local-first command-line tool that turns a plain-language biological question into code you can read and run, executes that code in a sandboxed Docker container, and records signed provenance for every step in a local database. It reads scientific literature, queries dozens of biological databases, and runs analyses in Python and R environments. It is model-agnostic: you bring your own key for Claude, an OpenAI-compatible endpoint, or a local model.

Inflexa is released under the permissive **Apache-2.0** license. Its open-source command-line tool is the full product, not a trial; the separate commercial offering only adds hosted infrastructure, team collaboration, and managed compute. As of August 2026 it is young but actively developed (around 30 stars), and its analysis "skill packs" are omics- and translational-medicine-focused (transcriptomics, proteomics, pharmacokinetics, drug repurposing, statistical modeling).

Useful to us? Yes, worth evaluating

Of the three outside tools, Inflexa is the most relevant to our actual research. Its priorities are ours: reproducible analysis in R and Python, provenance on every step, and no data leaving the machine. It is worth a real trial on a lab dataset, and its provenance and sandboxing design is a useful reference for our own reproducibility tooling. Two cautions: it is early, and its skill packs are aimed at omics rather than serology or infectious-disease epidemiology, so adopting it as a dependency now would be premature. Evaluate first; contribute a domain skill pack if the fit turns out to be good.

5 TORQCLAW

TORQCLAW (pilotwaffle 2026) is a "governed local and cloud AI-agent control plane." It is a TypeScript gateway, router, and console user interface wrapped around a forked Python execution engine, adding a governance layer between an agent's request and its action: approval gates, budget caps, spending receipts, and capability and path scoping over Model Context Protocol (MCP) tools. It runs local models through Ollama and falls back to a frontier model when needed.

As of August 2026 it is a single-author, early-stage project (a few stars, a completed "Phase 1" prototype). Critically, it ships **with no license file**, which under default copyright means all rights are reserved: we could not legally fork, vendor, or reuse its code even if we wanted to.

Useful to us? Not for adoption

TORQCLAW fails three ways for our purposes. It is unlicensed, so we cannot legally build on it. It is domain-agnostic agent *infrastructure*, not tooling for our research. And the governance ideas it centers on, approval gates, capability scoping, and cost limits, we already implement directly through Claude Code permissions and hooks, MCP scoping, and the `Workflow` tool's token budgets. At most, its design documents are worth skimming as a catalog of agent-governance patterns.

6 Comparison

The table below places the three outside tools against our current stack. "Relevance to us" is the bottom line and follows directly from the rows above it.

Dimension	Claude Code + `ai-config` (ours)	Agent Teams	Inflexa	TORQCLAW
What it is	Our current agent stack	Multi-session teammates	Computational-biology orchestrator	Governed agent control plane
Primary domain	General coding and research	General coding and research	Computational biology	General agent infrastructure
License	Reusable across our repos	Anthropic product	Apache-2.0 (permissive)	None (all rights reserved)
Maturity (2026)	In daily use	Experimental, off by default	Young, active	Early prototype, one author
Execution model	Subagents, `Workflow`, worktrees	Lead plus independent teammates	Sandboxed Docker, R and Python	TypeScript gateway, Python engine
Governance built in	Hooks, permissions, budgets	Inherits Claude Code permissions	Sandbox, signed provenance	Approval gates, budgets, receipts
Local-first	Yes	Yes	Yes	Yes (Ollama, cloud fallback)
Relevance to us	The baseline	Try it	Evaluate it	Pass; reference only

7 Recommendation

For the lab, as of August 2026:

- **Try Agent Teams** on a research or review task where parallel, arguing agents would beat one agent working alone. It is native to our tooling and the cheapest to experiment with, as long as the token cost is watched.
- **Evaluate Inflexa** on a real dataset. It is the most domain-aligned option and openly licensed, and even if we do not adopt it, its provenance-and-sandbox design informs our own reproducibility work.
- **Pass on TORQCLAW** for adoption. It is unlicensed, off-domain, and centered on governance we already have.

None of these replaces our current Claude Code and `ai-config` practice. Agent Teams extends it, Inflexa is a candidate research tool alongside it, and TORQCLAW is, for now, only a reference.

References

Anthropic. 2026. *Orchestrate Teams of Claude Code Sessions*. Documentation. https://code.claude.com/docs/en/agent-teams.

Inflexa. 2026. *Inflexa: The Open-Source Orchestrator for Computational Biology*. Software. https://github.com/inflexa-ai/inflexa.

pilotwaffle. 2026. *TORQCLAW: A Governed Local and Cloud AI-Agent Control Plane*. Software. https://github.com/pilotwaffle/TORQCLAW.